



**POLYTECHNIQUE
MONTRÉAL**

UNIVERSITÉ
D'INGÉNIERIE

INF3500 - Conception et réalisation de systèmes numériques

Laboratoire 4

Marc-Antoine MANNINGHAM
Beaurel FOHOM

Automne 2025

Implémenter un processeur à usage général sur FPGA

À la fin de ce laboratoire, vous devrez être capable de :

- Analyser le chemin des données d'un processeur à usage général afin d'estimer les ressources requises, déterminer les limites opérationnelles du processeur, proposer des changements architecturaux pour implémenter de nouvelles opérations, et donner le code VHDL modifié (B5)
- Proposer des modifications à l'unité de contrôle et à l'encodage des instructions pour ajouter de nouvelles instructions à un processeur à usage général, et donner le code VHDL modifié (B5)

Ce laboratoire s'appuie sur et complète le matériel suivant :

1. Les procédures utilisées et les compétences développées dans les laboratoires précédents.
2. La matière du thème 9 (Conception de processeurs à usage général).

! Évaluation

Comme ce laboratoire est plutôt long, le rendu que vous aurez à produire consistera à **compléter le *notebook* Jupyter** fourni (en plus de vos fichiers source VHDL). Vous n'avez donc pas besoin de remettre un PDF.

Préliminaires

Allumez votre PYNQ, connectez-vous au serveur Jupyter, créez un dossier (par exemple, INF3500-labo4), mettez-y votre *notebook* labo4.ipynb et ouvrez-le. La fin du *notebook* est à compléter en guise de rapport. Dans la partie

! Attention

Comme vous devrez travailler sur votre PYNQ, pensez bien à copier votre *notebook* dans votre dépôt Git régulièrement, et à y copier votre version finale avant la date de remise.

Liste des fichiers

Fichier	Contenu
src/PolyRISC_utils_pkg.vhd	Définit le package PolyRISC_utils qui contient les constantes, types et utilitaires pour PolyRISC
src/PolyRISC_v2.vhd	Implémentation du PolyRISC en VHDL
src/PolyRISC_ROM.vhd	Mémoire des instructions du PolyRISC
src/AXI_stream_ROM.vhd	Convertisseur AXI-Stream pour écrire la mémoire des instructions depuis un notebook Jupyter à travers PYNQ. <i>Vous n'avez pas à vous soucier de ce fichier, plus de détails techniques sont présents dans le notebook pour les curieux</i>
src/power_on_reset.vhd	Module qui réinitialise tous les composants au démarrage
src/PolyRISC_tb.vhd	Banc d'essai pour tester le bon fonctionnement du PolyRISC et du programme
src/top.vhd	<i>Top-level</i> du projet
src/fibonacci_n.txt	Fichier de vecteurs de test pour le banc d'essai (voir la partie Simulation)
src/fibonacci_res.txt	Fichier contenant les nombres de Fibonacci correspondant à chaque entrée présente dans src/fibonacci_n.txt
xdc/PYNQ-Z2-v1.0.xdc	Fichier de contraintes pour la PYNQ-Z2
synth-impl/create_zynq_wrapper.tcl synth-impl/zynq.tcl	Scripts de génération du <i>wrapper</i> pour le Zynq, à sourcer après avoir importé les sources

Partie 0 : Introduction

Dans cet exercice de laboratoire, on considère l'implémentation d'un processeur à usage général modélisé en VHDL. Des modèles de processeurs ouverts existent et peuvent être analysés, simulés, modifiés, adaptés et implémentés par les utilisateurs dans la technologie de leur choix. Un exemple très populaire actuellement est le modèle d'architecture de processeur (ou de jeu d'instructions) [RISC-V](#). Dans ce laboratoire, nous utiliserons l'implémentation PolyRISC, développée expressément pour les besoins du cours INF3500.

Le but de ce laboratoire sera pour vous de comprendre le fonctionnement du PolyRISC, puis de lui ajouter des fonctionnalités, et enfin d'écrire un programme en langage machine pour un processeur RISC-V.

Retour sur le PolyRISC

Le processeur PolyRISC a une architecture à jeu d'instructions réduit (RISC : *Reduced Instruction Set Computer*). Il comporte un chemin des données formé d'un bloc des registres, d'une UAL et d'une mémoire des données (bien qu'on l'inclut ici, la mémoire des données n'est pas normalement considérée comme faisant partie du processeur). L'unité de contrôle est formée d'un compteur de programme, d'une mémoire des instructions, d'un bloc de décodage et d'une unité de branchement. Les diapositives et vidéos du thème 9 donnent tous les détails.

Le processeur PolyRISC inclut un registre d'entrée `i_GPIO` et un registre de sortie `o_GPIO` (GPIO : *General Purpose Input-Output*). Ces registres établissent des liens directs entre le monde extérieur et le chemin de données du processeur. Ils sont associés chacun avec un port de contrôle, respectivement `i_GPIO_valide` et `o_GPIO_valide`. (Attention, la sortie `o_GPIO_valide` passe par une bascule qui n'est pas indiquée sur le schéma.)

Attention, le schéma de la figure 1 montre une mémoire interne, mais pour les besoins de ce laboratoire, la mémoire est externe. Ainsi, le PolyRISC a aussi un port de sortie `o_inst_addr` sans registre, directement piloté par son compteur de programme, et une entrée `i_inst` pour recevoir l'instruction stockée dans la mémoire à l'adresse indiquée.

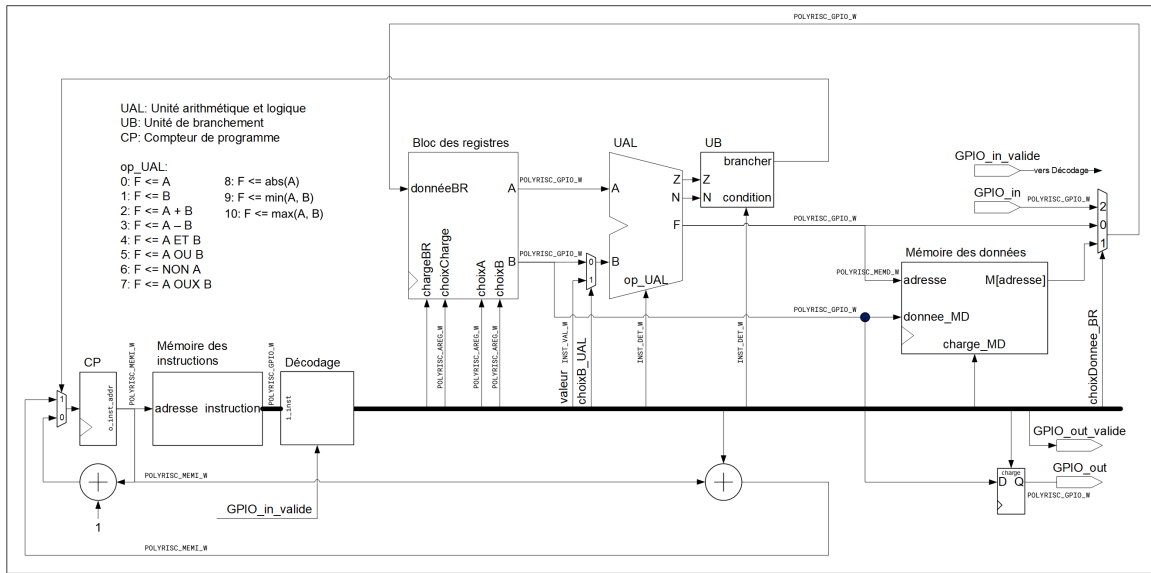


Fig. 1. – Schéma d'architecture du PolyRISC

Pour les besoins de cet exercice, nous considérons le cas où le PolyRISC a un chemin des données de 32 bits de large, un bloc de 16 registres et une mémoire des données de 256 mots. La mémoire des instructions a aussi une capacité de 256 mots de 32 bits¹.

Le processeur et sa mémoire d'instructions sont décrits avec trois fichiers :

- `src/PolyRISC_utils_pkg.vhd` : Un package en VHDL qui définit les constantes (largeur différents chemins, nombre de registres, etc.), les types ainsi qu'un certain nombre de fonctions utilitaires. Notamment, ce fichier définit le type `instruction_t` qui permet de rendre la manipulation des instructions RISC-V plus accessible à un humain, à l'aide d'un record en VHDL (similaire à `struct` en C)
- `src/PolyRISC_v2.vhd` : La description principale des chemins de données et de contrôle de PolyRISC. Prenez le temps de parcourir ce fichier pour relier les différents processus et signaux VHDL et les parties correspondantes du schéma
- `src/PolyRISC_ROM.vhd` : La description de la mémoire des instructions. Comme vous le verrez dans le notebook relié à ce laboratoire, elle est utilisée en lecture seule par le PolyRISC mais peut être écrite depuis PYNQ à travers une communication AXI. *Comprendre le fonctionnement en écriture de la mémoire n'est pas nécessaire pour finir ce laboratoire.*

Nous vous invitons à consulter ces fichiers pour au moins comprendre les grandes lignes de l'implémentation.

Un banc d'essai très rudimentaire, `src/PolyRISC_tb.vhd`, est fourni pour vérifier le bon fonctionnement du processeur et d'un programme.

¹Contrairement au modèle vu en classe qui avait des adresses de 32 bits, on utilise ici seulement les 8 bits de poids le plus faible du chemin de données pour adresser la mémoire des données.

Programme de démonstration : nombres de Fibonacci

Un programme de calcul du n -ième nombre de la suite de Fibonacci, F_n , vous est fourni à titre d'exemple. Il est défini en langage machine dans le banc d'essai `PolyRISC_tb.vhd` et dans le notebook Python, pour vous donner un exemple de comment vous testerez votre programme, en simulation puis en conditions réelles. Ce programme correspond au pseudo-code 1 (le signe « dollar » (\$) indique une constante).

```
R0 := GPIO_in
R1 := $0
R3 := $0
R4 := $1
si R1 = R0 aller à CP + 6
R5 := R3 + R4
R3 := R4
R4 := R5
R1 := R1 + $1
si R1 <= R0 aller à CP + -4
GPIO_out := R3
```

Pseudo-code 1. – Calcul des nombres de Fibonacci en pseudo-assembleur

Dans ce programme, les registres 0 à 5 du PolyRISC sont donc utilisés comme indiqué dans le tableau suivant :

Registre	Contenu
fichierRegistres[0]	n : entrée
fichierRegistres[1]	k : rang courant dans la suite de Fibonacci
fichierRegistres[2]	pas utilisé
fichierRegistres[3]	nombre F_n
fichierRegistres[4]	nombre F_{n+1}
fichierRegistres[5]	accumulateur

Tableau 1. – Utilisation des registres dans le programme exemple

Simulation

Pour valider le bon fonctionnement du PolyRISC et du programme de démonstration, vous pouvez le simuler avec le fichier `src/PolyRISC_tb.vhd`. La simulation implémentée ici lit ses vecteurs de test dans un fichier, `src/fibonacci_n.txt` : ce laboratoire a aussi pour but de vous présenter la manipulation de fichiers en VHDL. Lorsque vous lancerez la simulation, le banc d'essai lira ses entrées dans `src/fibonacci_n.txt` et écrira ses valeurs de sortie sur sa sortie standard (elles seront reportées dans la console TCL sous Vivado).

! Attention

Vous devez faire en sorte que le fichier de vecteurs de test soit accessible à la simulation. Pour ce faire, vous avez deux choix :

- Placer le fichier `fibonacci_n.txt` dans le même dossier que l'exécutable de la simulation (Sous Vivado, c'est `<projet>/<nom_projet>.sim/sim_1/behav/xsim/` par défaut). Dans ce cas, il faudra lancer la simulation une première fois pour créer ce répertoire, et la simulation terminera avec un message d'erreur indiquant que `fibonacci_n.txt` n'a pas été trouvé. Vous pourrez alors mettre le fichier dans le répertoire nouvellement créé et relancer la simulation.
- Modifier le banc d'essai `PolyRISC_tb.vhd` en remplaçant « `fibonacci_n.txt` » par le chemin complet du fichier dans la commande `file_open` (ligne 108)

! Évaluation

Ajoutez une capture d'écran de votre simulation à votre *notebook*. *N'oubliez pas d'ajouter votre capture à votre dépôt Git !*

Analyse des résultats de simulation

Vous pouvez naviguer de transition en transition sur le signal de votre choix dans la fenêtre de visualisation des signaux en le sélectionnant et en utilisant les flèches droite et gauche (cela peut servir pour naviguer directement sur les transitions de `o_GPIO_valide` par exemple, comme la simulation est assez longue.)

Si tout s'est bien passé, vous devriez obtenir les mêmes sorties, et dans le même ordre, que celles présentes dans `src/fibonacci_res.txt`. **Enfin, presque.** Le dernier nombre affiché par la simulation est négatif. Pourtant, le programme et le PolyRISC fonctionnent parfaitement bien.

! Évaluation

Expliquez dans la partie 0 du *notebook* pourquoi le dernier nombre de Fibonacci retourné par PolyRISC est négatif. Justifiez vos explications avec une démonstration dans votre *notebook*.

Manipulation de fichiers en VHDL

Ce fonctionnement ne permet pas de sauvegarder les résultats efficacement, pour pouvoir les réutiliser ou automatiser leur vérification. **C'est pourquoi vous allez implémenter l'écriture d'un fichier avec les résultats dans le banc d'essai.**

! Évaluation

Utilisez les fonctions VHDL `file_open`, `write`, `writeline` et `file_close` pour écrire les sorties du banc d'essai dans un fichier `.txt`. Vous pouvez vous inspirer de la méthode utilisée pour lire dans le fichier de vecteurs. Ajoutez votre fichier de sortie pour l'algorithme de calcul des nombres de Fibonacci à votre dépôt, sous le nom `fibonacci_out.txt`. *N'oubliez pas de push vos modifications.*

Validation en conditions réelles sur la carte

Une fois votre système testé et validé en simulation, vous pouvez le synthétiser, l'implémenter, et générer un *bitstream*. Ensuite, vous allez tester le programme de démonstration.

Pour ce faire, vous devez copier **deux fichiers** dans la PYNQ : le *bitstream* et le *hardware handoff*. Le *bitstream* (`.bit`) est le fichier habituel généré par Vivado à l'issue des différentes étapes de synthèse, implémentation, placement et routage. C'est le fichier qui est utilisé pour programmer le FPGA. Le **hardware handoff** (`.hwh`) dépasse largement le cadre de ce cours, mais il sert en gros au *framework* PYNQ pour reconnaître la configuration matérielle du design et pouvoir ainsi se paramétrer pour communiquer avec.

Par défaut,

- `top.bit` est sous `<projet>/<nom_projet>.runs/impl_1/top.bit`
- `zynq.hwh` est sous `<projet>/<nom_projet>.gen/sources_1/bd/zynq/hw_handoff/zynq.hwh`

! Attention

Dans le *notebook*, l'appel à `pynq.Overlay` s'attend à ce que les deux fichiers `.bit` et `.hwh` portent le même nom avant extension, `labo4`. Vous devrez donc les renommer à chaque fois que vous mettrez à jour votre design². Ce sera la première chose à vérifier si un design qui simule correctement ne fonctionne pas une fois implémenté.

Une fois ces deux fichiers copiés dans le même dossier que votre *notebook*, rendez vous à la partie « Exemple : suite de Fibonacci » et exécutez toutes ses cellules (après avoir exécuté les cellules au-dessus ; il y a une option « Execute all cells above » dans l'interface web). Pour les plus curieux, vous pouvez lire les parties précédentes, mais ce n'est pas nécessaire pour mener à bien le laboratoire. Il suffit pour cela de comprendre le rôle des trois cellules de cette partie.

²Normalement, vous n'aurez pas à le faire trop souvent car ce laboratoire est surtout axé sur l'écriture d'instructions RISC

Partie 1 : Approximation de racine carré³ par recherche dichotomique

Votre travail dans ce laboratoire consiste à implémenter une recherche dichotomique en langage machine pour le PolyRISC.

Partie 1a. : Pseudocode

L'algorithme d'approximation de la racine carré par recherche dichotomique est donné par le pseudo-code 2 (remerciements au professeur [Jean Pierre David](#)).

Notez que tel qu'implémenté, cet algorithme peut seulement déterminer une approximation pour des valeurs dont la racine s'exprime sur 16 bits en signé, c'est-à-dire des valeurs inférieures à $0x7FFF^2 = 32767^2 = 1073676289 = 0x3FFF0001$

Une feuille de calcul est disponible à [ce lien](#) pour vous permettre d'expérimenter différentes valeurs en entrée et d'observer le déroulement du calcul. Créez une copie locale du fichier pour l'éditer. La table 2 montre un exemple de déroulement de l'algorithme pour l'entrée 7895.

```
rechercheDichotomique(nombre) {
  nombre := entrée externe;
  haut   := 0x7FFF; -- max sur 16 bits, signé
  bas    := 0;
  compteur := 16;
  tant que compteur > 0
  {
    pivot := (haut + bas) / 2;
    carre := pivot × pivot;
    si carre > nombre
    {
      haut := pivot;
    }
    sinon
    {
      bas := pivot;
    }
    compteur := compteur - 1;
  }
  retourner pivot;
```

Pseudo-code 2. – Algorithme de recherche dichotomique

³Oui, on écrit « carré » et pas « carrée » car il ne s'agit pas d'une racine de forme carrée, mais de la racine de la fonction qui à un nombre associe son carré. Je n'en démordrai pas.

compteur	bas	haut	pivot	carre
16	0	32767	non init.	non init.
15	0	16383	16383	268402689
14	0	8191	8191	67092481
13	0	4095	4095	16769025
12	0	2047	2047	4190209
11	0	1023	1023	1046529
10	0	511	511	261121
9	0	255	255	65025
8	0	127	127	16129
7	63	127	63	3969
6	63	95	95	9025
5	79	95	79	6241
4	87	95	87	7569
3	87	91	91	8281
2	87	89	89	7921
1	88	89	88	7744
0	88	89	88	7744

Tableau 2. – Exemple de déroulement de l'algorithme pour l'entrée 7985

Il n'est pas possible de programmer le PolyRISC pour implémenter cet algorithme : il manque la fonction de division par 2. Par ailleurs, même si on pourrait réaliser la multiplication par des additions successives, ce serait plutôt lourd. **Vous allez donc devoir modifier le PolyRISC pour lui ajouter les instructions nécessaires.**

Partie 1b. Ajout des instructions $RC := RA \times RB$ et $RC := RA \times \text{valeur}$ au PolyRISC

On commence par ajouter la fonctionnalité de multiplication à l'unité arithmétique et logique (UAL) du PolyRISC⁴.

La multiplication de deux nombre sur N bits génère un produit sur maximum $2N$ bits. Comme la largeur des données du PolyRISC est définie par `POLYRISC_GPIO_W`, vous allez donc réaliser la multiplication des `POLYRISC_GPIO_W` / 2 bits de poids le plus faible de `RA` et `RB`, pour s'assurer que le résultat ne dépasse pas la largeur de `RC`.

⁴Contrairement à l'exemple vu en classe, le détail de l'instruction est donc ici encodé sur 4 bits, et non 3

Rappel : sélection de bits en VHDL

Pour sélectionner les N bits les moins significatifs de l'objet A défini par
 $A : \text{unsigned}(M - 1 \text{ downto } 0)$ (avec $M > N$), on utilise $A(N - 1 \text{ downto } 0)$

Observez le code du processeur dans `src/PolyRISC_v2.vhd` pour comprendre comment l'UAL est implémentée. Faites le lien avec les constantes définies dans `src/PolyRISC_utils_pkg.vhd`. Pour ajouter votre instruction de multiplication, commencez par ajouter une constante `AmulB` dans `src/PolyRISC_utils_pkg.vhd` et donnez-lui la valeur 11. Implémentez ensuite la fonctionnalité dans `src/PolyRISC_v2.vhd`.

Notez bien que grâce au design modulaire de RISC-V, l'ajout de cette opération à l'UAL permet d'implémenter d'un coup les instructions RC
 $:= RA \times RB$ et $RC := RA \times \text{valeur}$.

Évaluation

Ajoutez la fonctionnalité de multiplication à l'UAL du PolyRISC. Respectez au maximum les conventions que vous observez dans les fichiers source. *N'oubliez pas de push vos modifications.*

Attention

Conservez une copie inchangée de `src/PolyRISC_v2.vhd` tel qu'on vous l'a fourni : elle vous sera utile pour les calculs d'utilisation de ressources de la partie 3.

Pour vous assurer de n'avoir rien brisé, **relancez la simulation** avec le programme de calcul des nombres de Fibonacci. Vous pouvez aussi être vraiment sûr.e.s en relançant le code Python dans le *notebook* avec votre nouveaux *bitstream* et *hardware handoff*.

Partie 1c. Ajout de l'instruction $RC := RA / 2$ au PolyRISC

Évaluation

En procédant de la même manière que pour la partie précédente, ajoutez l'instruction $RC := RA / 2$ au PolyRISC. Pour ce faire, utilisez une constante `Adiv2` avec la valeur 12 dans `src/PolyRISC_utils_pkg.vhd`. *N'oubliez pas de push vos modifications.*

Notez bien qu'il s'agit ici d'une division entière, puisque le PolyRISC ne comprend pas les nombres flottants.

Pour vous assurer de n'avoir rien brisé, **relancez à nouveau la simulation** avec le programme de calcul des nombres de Fibonacci. Vous pouvez aussi être vraiment sûr.e.s

en relançant le code Python dans le *notebook* avec votre nouveaux *bitstream* et *hardware handoff*. Enfin, pour bien vérifier que tout se passe bien, vous pouvez comparer les deux fichiers de sorties des simulations avec un logiciel de comparaison (par exemple, `diff`) et vérifier qu'ils sont identiques.

Partie 1d. : Implémentation et simulation de l'algorithme de recherche dichotomique en langage machine

Implémentation

Dans cette partie, vous devez implémenter le pseudo-code 2 pour le PolyRISC. Commencez par l'écrire en pseudo-assembleur comme le programme de calcul des nombres de Fibonacci (voir le pseudo-code 1), en utilisant vos nouvelles opérations $RC := RA \times RB$ et $RC := A / 2$.

! Évaluation

Écrivez votre implémentation de l'algorithme de recherche dichotomique en pseudo-assembleur dans votre *notebook*. Écrivez aussi quel registre est utilisé pour quelle fonction dans votre implémentation, à la manière de l'exemple donné avec l'algorithme de calcul des nombres de Fibonacci.

Simulation

Une fois votre programme écrit, vous devrez le simuler pour vérifier son bon fonctionnement. Pour ce faire, remplacez la valeur de la constante `memoire_inst` dans `src/PolyRISC_tb.vhd` par votre programme. Pour vous faciliter la tâche, `gen_inst` génère une instruction (de type `instruction_t`) à partir des différents champs passés avec des valeurs entières. N'oubliez pas d'appeler `inst_to_unsigned` pour convertir ensuite l'instruction générée en `unsigned` (prenez exemple sur la définition du programme de calcul des nombres de Fibonacci qui vous est fournie dans le banc d'essai). Modifiez ensuite vos fichiers d'entrée et de sortie en `dichotomie_vecs.txt` et `dichotomie_out.txt` respectivement. Créez vos vecteurs de test en écrivant **au moins 20 valeurs** entre 0 et `0x3FFF0001` (1073676289) dans le fichier de vecteurs `dichotomie_vecs.txt`, une par ligne.

Automatisez la vérification de vos résultats. Au choix, vous pouvez :

- Ajouter des énoncés `assert` dans le banc d'essai en utilisant `MATH_REAL.FLOOR` et `MATH_REAL.SQRT` pour vérifier vos solutions. Reportez-vous à la feuille de calcul pour voir quel résultat est attendu.
- Écrire un script (en Python, `bash` ou autre⁵) qui lit votre fichier de sorties et vérifie les résultats. Dans ce cas, pensez à ajouter votre script dans votre dépôt Git.

⁵Ça peut être n'importe quoi du moment qu'on peut le lancer soit avec Vivado, soit depuis un terminal UNIX avec les paquets de base installés.

! Évaluation

Faites un *push* de vos modifications faites sur le banc d'essai, avec votre programme à la place du programme d'exemple. Ajoutez aussi votre fichier de vecteurs de test, votre fichier de sorties de banc d'essai à votre dépôt et votre script de validation le cas échéant.

Partie 2 : Implémentation matérielle

Pour tester votre PolyRISC, vous allez devoir utiliser le *framework* PYNQ pour établir une communication entre le processeur ARM de la carte et le PolyRISC. Comme au laboratoire précédent, nous allons donc utiliser la classe `Overlay` de `pynq` pour programmer le Zynq, au lieu de le faire directement depuis Vivado.

Partie 2a. Utilisation de PYNQ

Pour tester votre système amélioré, procédez de la même façon qu'avec l'exemple de la partie 1. Après avoir copié vos nouveaux fichiers `.bit` et `.hwh` sur la PYNQ, **ré-exécutez le test du programme d'exemple pour vous assurer que tout fonctionne toujours correctement.**

! Évaluation

Ajoutez votre nouveau *bitstream* ainsi que votre *hardware handoff* fonctionnels dans le dossier `synth-impl` de votre dépôt Git.

Une fois que cette partie sera validée, vous n'aurez plus à re-programmer le FPGA la PYNQ, mais seulement à écrire du langage machine.

Partie 2b. Implémentation en langage machine

Comme expliqué dans le *notebook*, l'utilisation de PYNQ permet de réécrire un programme différent dans la mémoire des instructions sans avoir à re-synthétiser l'intégralité du design, au moyen d'un buffer exposé sous la forme d'un *array numpy*.

Vous devez donc à présent implémenter votre programme de recherche dichotomique directement en langage machine. Si votre simulation a fonctionné correctement, il vous suffit de convertir chaque instruction en nombre binaire sur 32 bits, en concaténant chaque champ⁶. Inspirez-vous de l'exemple donné dans le *notebook*.

⁶C'est aussi littéralement ce qui est fait par la fonction `inst_to_unsigned` du package `PolyRISC_utils`

Conseil : notation binaire en Python

En Python, comme dans la plupart des langages de programmation, on peut écrire des littéraux directement en binaire avec le préfixe `0b`. De plus, insérer un *underscore* (`_`) ou des `0` en préfixe n'a aucun effet sur la valeur du nombre, cela permet simplement de rendre le code plus lisible.

Par exemple, toutes les lignes suivantes sont équivalentes :

```
a = 52
a = 0b110100
a = 0b11_01_00
a = 0b0011_0100
```

Pour tester votre programme, utilisez les fonctions `flash` et `test_PolyRISC`, dans cet ordre. `flash` écrit la liste d'instructions passée en argument dans la mémoire des instructions du PolyRISC, et `test_PolyRISC` envoie une entrée au PolyRISC et retourne sa sortie.

Évaluation

Complétez la partie 2 du *notebook* Jupyter en implémentant votre programme de recherche dichotomique en langage machine. *N'oubliez pas de le mettre à jour sur votre dépôt Git !*

Partie 3 : ressources pour implémenter le PolyRISC

Dans cette dernière partie, vous allez mesurer l'utilisation des ressources pour synthétiser le PolyRISC dans différents cas de figure. Vous allez comparer votre version avec deux instructions supplémentaires et la version de base de PolyRISC.

Le tableau 3 résume les différents cas que vous devez tester. Pour chaque cas, modifiez les valeurs des constantes dans `src/PolyRISC_v2.vhd` et lancez la synthèse. Récupérez ensuite les résultats d'utilisation de **Slice LUT** dont **LUT as Memory**, **Slice Registers**, **F7 Muxes**, **F8 Muxes** et **Bonded IOB** pour compléter le tableau du *notebook*.

Info

Pour synthétiser uniquement un module défini par l'entité PolyRISC, vous pouvez utiliser la commande

```
synth_design -top PolyRISC -part xc7z020clg400-1 -assert
```

Le rapport d'utilisation des ressources est ensuite obtenu avec

```
report_utilization [-file <fichier> [-append]]
```

- L'option `-file` spécifie un fichier dans lequel Vivado doit écrire son rapport d'utilisation. En son absence, le rapport sera écrit dans la console TCL.
- Si l'option `-append` est passée avec `-file`, le rapport est ajouté à la fin de `<fichier>`. Sinon, `<fichier>` est réécrit avec le nouveau rapport.

Si Vivado vous envoie des avertissements concernant le timing du système, ignorez-les ; le but ici est uniquement d'observer l'évolution de la consommation de ressources.

POLYRISC_NREG	POLYRISC_GPIO_W	Version
16	32	base
32	32	base
16	64	base
32	64	base
16	32	votre version
32	32	votre version
16	64	votre version
32	64	votre version

Tableau 3. – Cas à implémenter pour rapporter l'utilisaton de ressources

Évaluation

Complétez la partie 3 du *notebook* avec la quantité de **Slice LUT**, **LUT as Memory**, **Slice Registers**, **F7 Muxes** et **F8 Muxes** utilisés **par l'instance du PolyRISC uniquement**, que vous relevez pour chaque cas (précisez l'unité utilisée). Ajoutez un commentaire sur l'évolution des ressources consommées (notamment des *LUT as Memory*) en fonction des valeurs des différents paramètres et de la version du processeur considérée. *N'oubliez pas de mettre à jour votre notebook sur votre dépôt !*

Partie 4 : Défi

Mise en garde. Compléter correctement les parties précédentes peut donner une note de 17 / 20 (85%), ce qui peut normalement être interprété comme un A. La partie 4 demande du travail supplémentaire qui sort normalement des attentes du cours. Il n'est pas nécessaire de la compléter pour réussir le cours ni pour obtenir une bonne note. Il n'est pas recommandé de s'y attaquer si vous éprouvez des difficultés dans un autre cours. La partie 4 propose un défi supplémentaire pour les personnes qui souhaitent s'investir davantage dans le cours INF3500 en toute connaissance de cause.

Pour finir, le défi consiste à implémenter un algorithme de votre choix en pseudo-code, puis en langage machine pour le faire tourner sur votre PYNQ. Pour vous assurer qu'il fonctionne correctement, simulez-le comme vous l'avez fait dans les parties précédentes, puis incluez votre liste d'instructions machine dans votre *notebook*.

! Attention

Dans votre dépôt final, le fichier `PolyRISC_tb.vhd` doit contenir votre implémentation de la recherche dichotomique. Si vous choisissez de faire cette partie, pensez donc à rétablir l'état de votre dépôt Git tel qu'il était à la fin de la partie précédente **avant la date limite**.

! Évaluation

Implémentez un algorithme de votre choix, vérifiez-le et validez-le sur la carte en complétant la partie 4 du *notebook*.

Remise

La remise se fait directement sur votre dépôt Git. Faites un *push* régulier de vos modifications, et faites un *push* final avant la date limite de la remise. Respectez l'arborescence de fichiers originale. Consultez le barème de correction pour la liste des fichiers à remettre.

Directives spéciales :

- Ne modifiez pas les noms des fichiers, entités ou architectures, ni les listes de **generics** ou de ports
- Remettez du code de très bonne qualité, lisible, bien indenté et commenté.
- Indiquez clairement la source de tout code que vous réutilisez ou dont vous vous inspirez.
- **Pour ce laboratoire uniquement :** le *notebook* à compléter constitue votre rapport

Barème de correction

Le barème de correction est progressif. Il est relativement facile d'obtenir une note de passage (> 10) au laboratoire et il faut mettre du travail pour obtenir l'équivalent d'un A (17/20). Obtenir une note plus élevée (jusqu'à 20/20) nécessite plus de travail que ce qui est normalement demandé dans le cadre du cours et plus que les 9 heures que vous devez normalement passer par thème sur ce cours.

Critères	Points
Partie 0 : Simulation	
• Votre capture de simulation	0.5
• Votre banc d'essai PolyRISC_tb.vhd modifié pour écrire les sorties dans un fichier texte et votre fichier fibonacci_out.txt	2
• Pourquoi PolyRISC donne un F_{56} négatif ?	1
Partie 1 : Recherche dichotomique	
• Votre fichier PolyRISC_v2.vhd avec la multiplication ajoutée (1b.)	1
• Votre fichier PolyRISC_v2.vhd avec la division entière par 2 ajoutée (1c.)	1
• Votre implémentation de la recherche dichotomique en pseudo-assembleur dans le <i>notebook</i> (1d.)	4
• Votre fichier PolyRISC_tb.vhd modifié pour tester votre programme, vos fichiers textes dichotomie_vecs.txt et dichotomie_out.txt (1d.)	2
Partie 2 : Implémentation	
• Votre implémentation matérielle sous la forme de vos fichiers .bit et .hwh	0.5
• Votre programme en langage machine dans le <i>notebook</i>	3
Partie 3 : Utilisation de ressources	
• Vos résultats d'utilisation de ressources pour les 8 cas demandés	2
Sous-total	17
Partie 4 : Défi	
• Votre algorithme est correctement implémenté en pseudo-assembleur	1
• Votre algorithme est correctement implémenté en langage machine	1
• Votre algorithme fonctionne (et est original)	1
Total	20